

Efecte de iluminare

Cristian Lambru: 2025-26





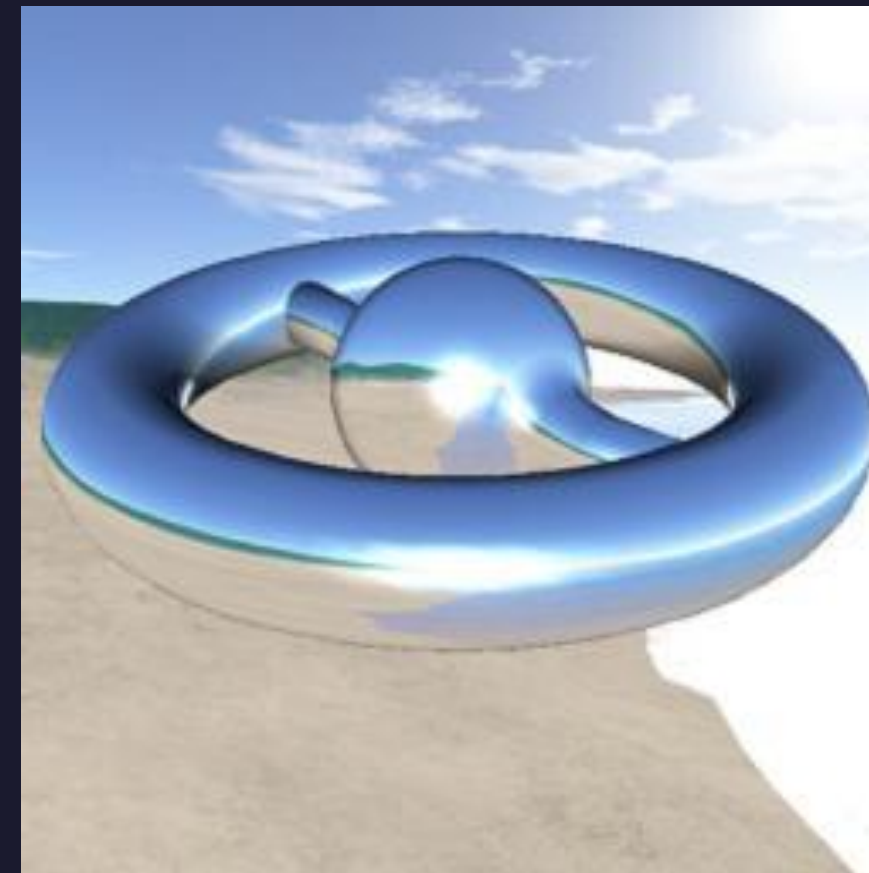
Cuprins

1. Tehnica de cartografiere a mediului înconjurător - „*Environment Mapping*”;
2. Tehnici în spațiul ecranului;
 - a. Tehnica de obținere a reflexiilor - „*Screen Space Reflections*”
 - b. Tehnica de calculare a obturării componentei ambientale - „*Ambient Occlusion*”



Cartografierea mediului înconjurător (I)

Tehnică propusă în anul 1976 de James Blinn pentru simularea efectului de *reflexii ale luminii pe suprafețe lucioase*.

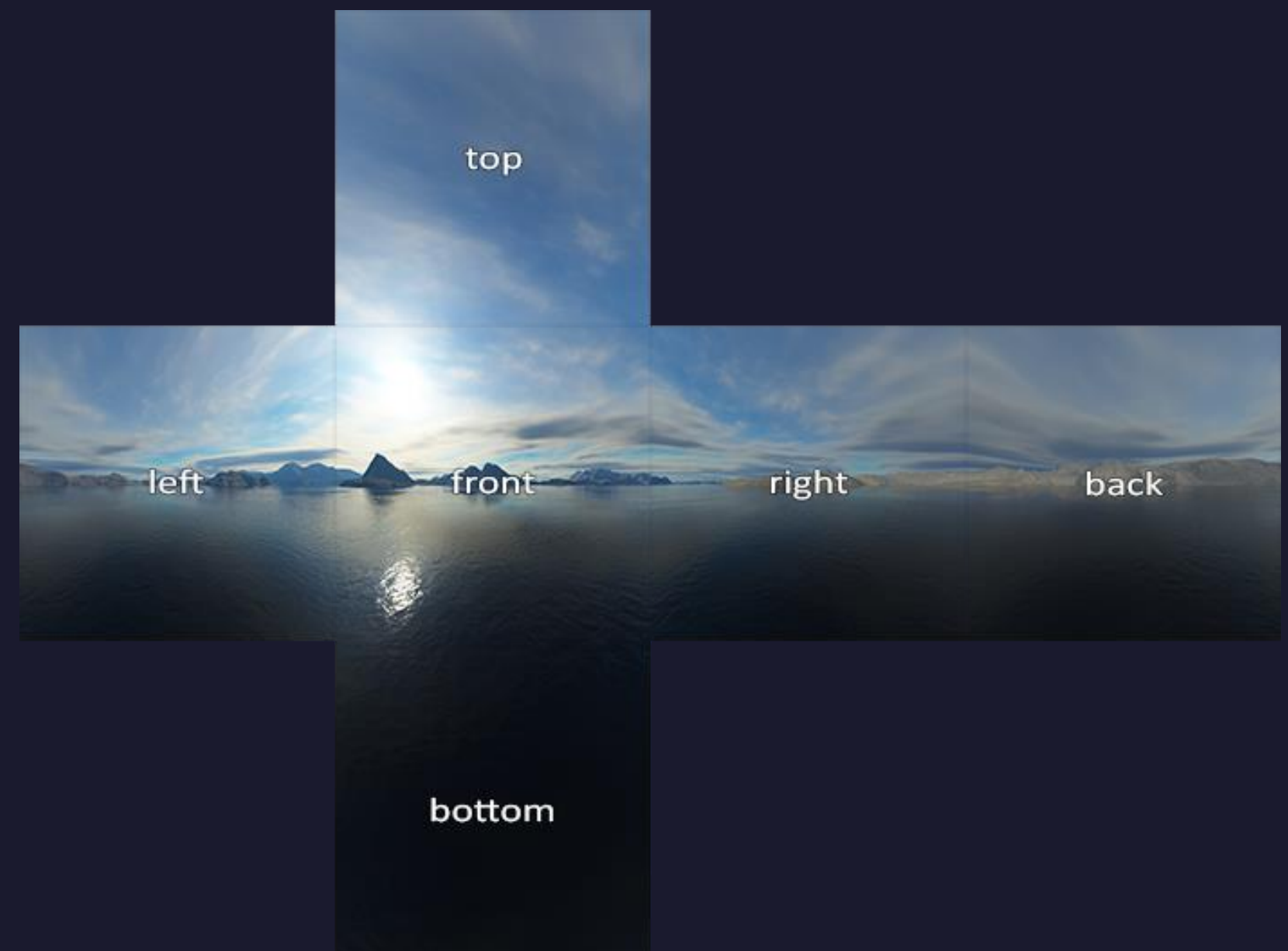


Blinn, James F., and Martin E. Newell. "Texture and reflection in computer generated images." Communications of the ACM 19.10 (1976): 542-547.



Cartografierea mediului înconjurător (II)

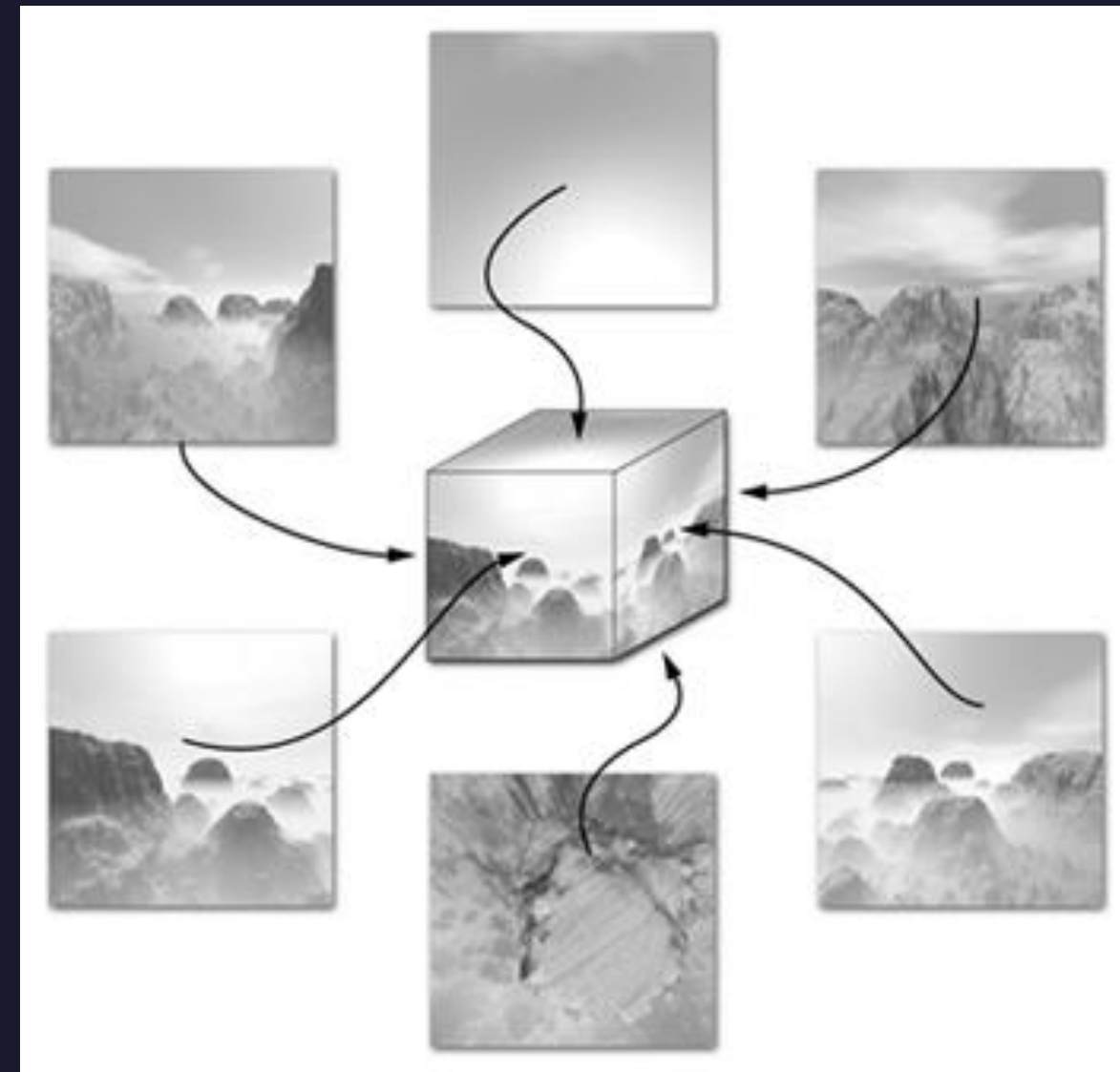
Se utilizează un obiect de tip textură cubică, care cartografiază fețele unui cub - „*Cube Map Texture*”.

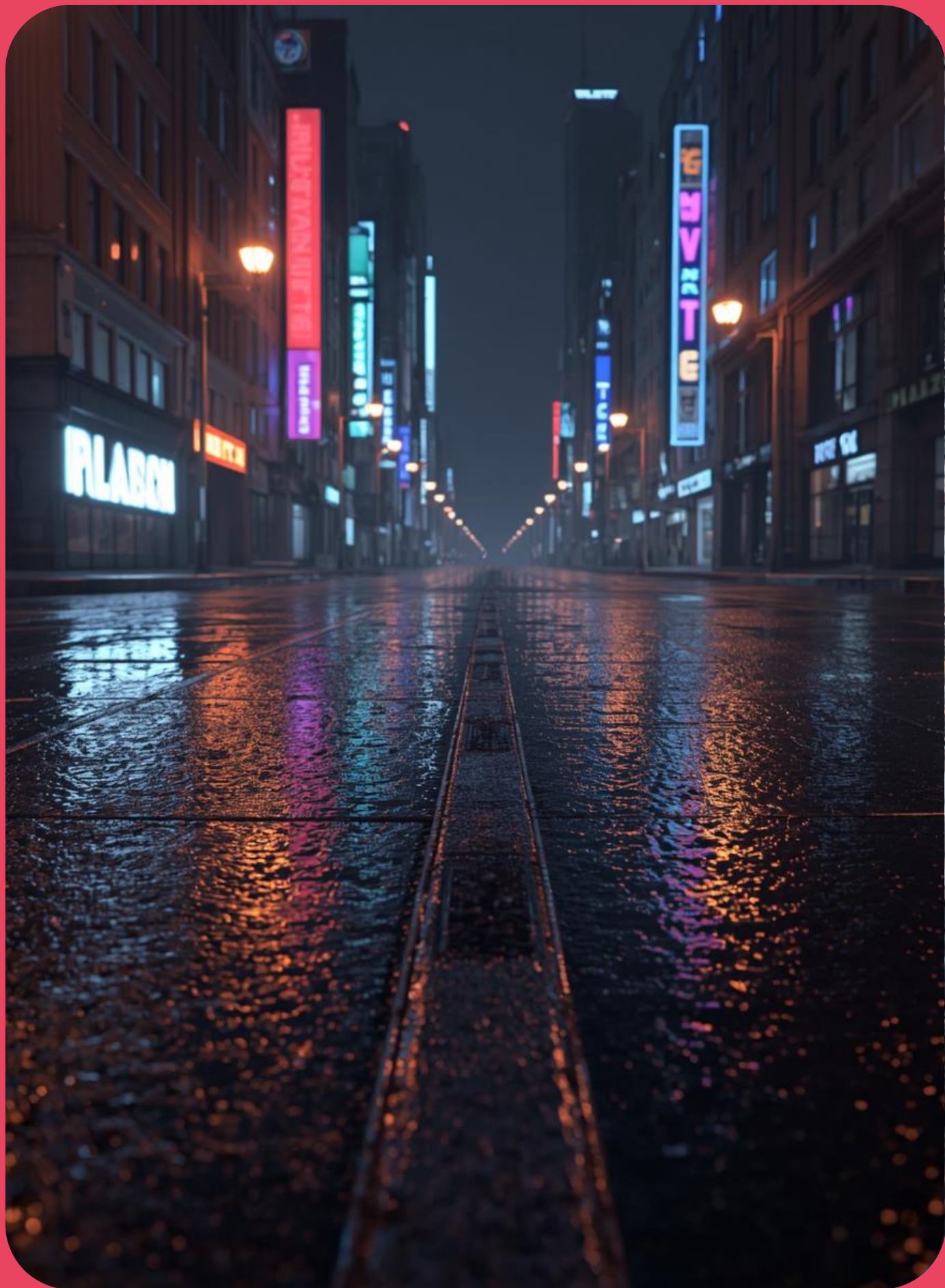




Cartografierea mediului înconjurător (III)

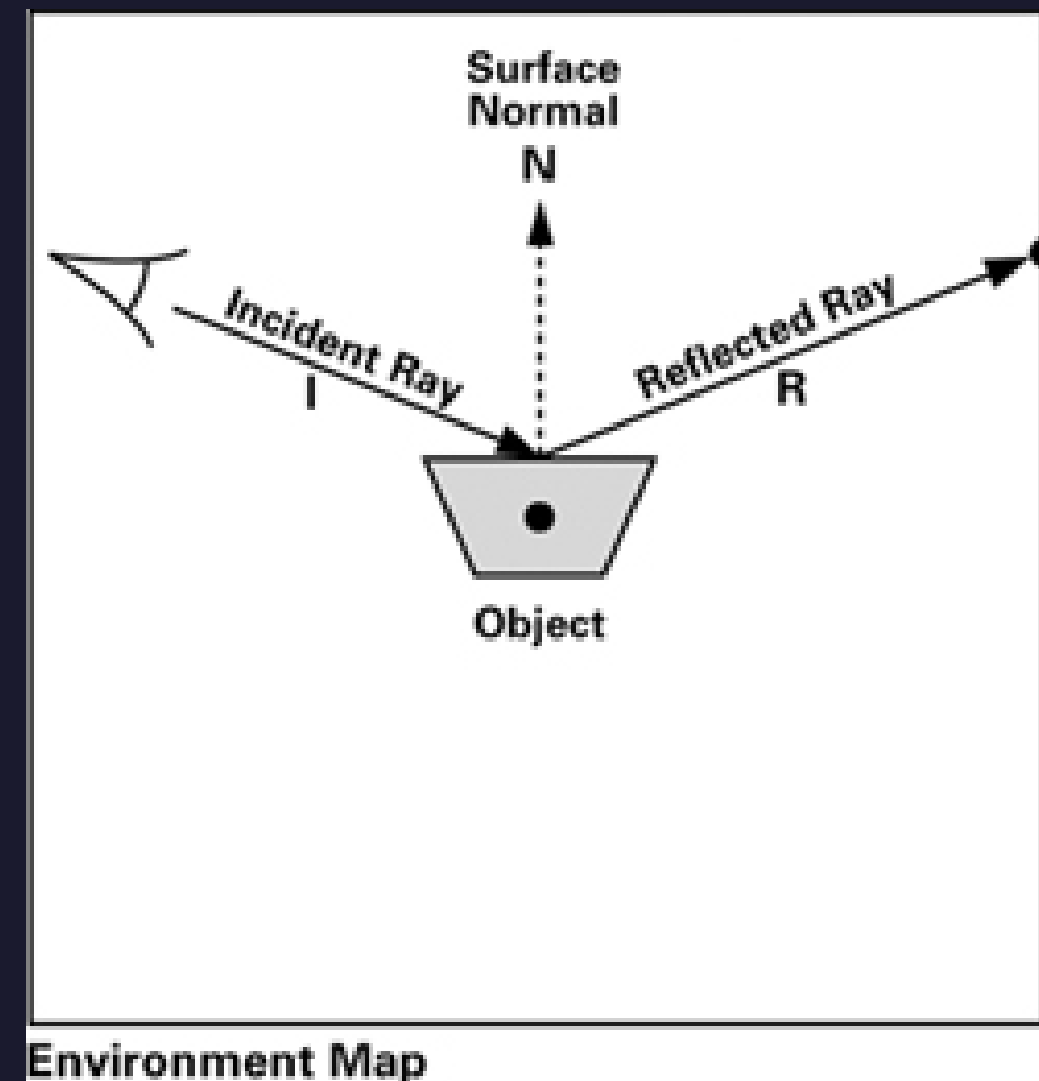
Se utilizează un obiect de tip textură cubică, care cartografiază fețele unui cub - „*Cube Map Texture*”.

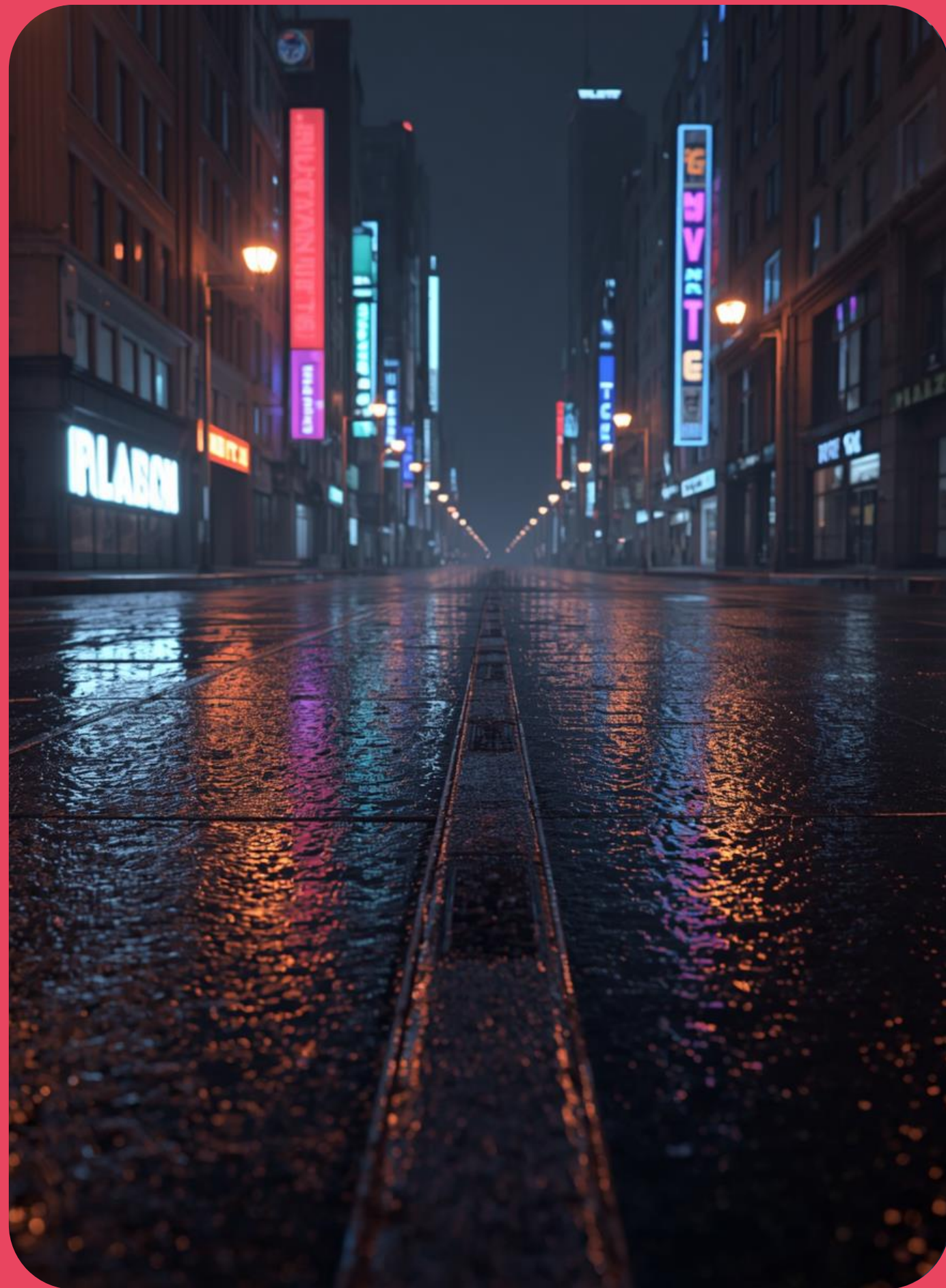




Cartografierea mediului înconjurător (IV)

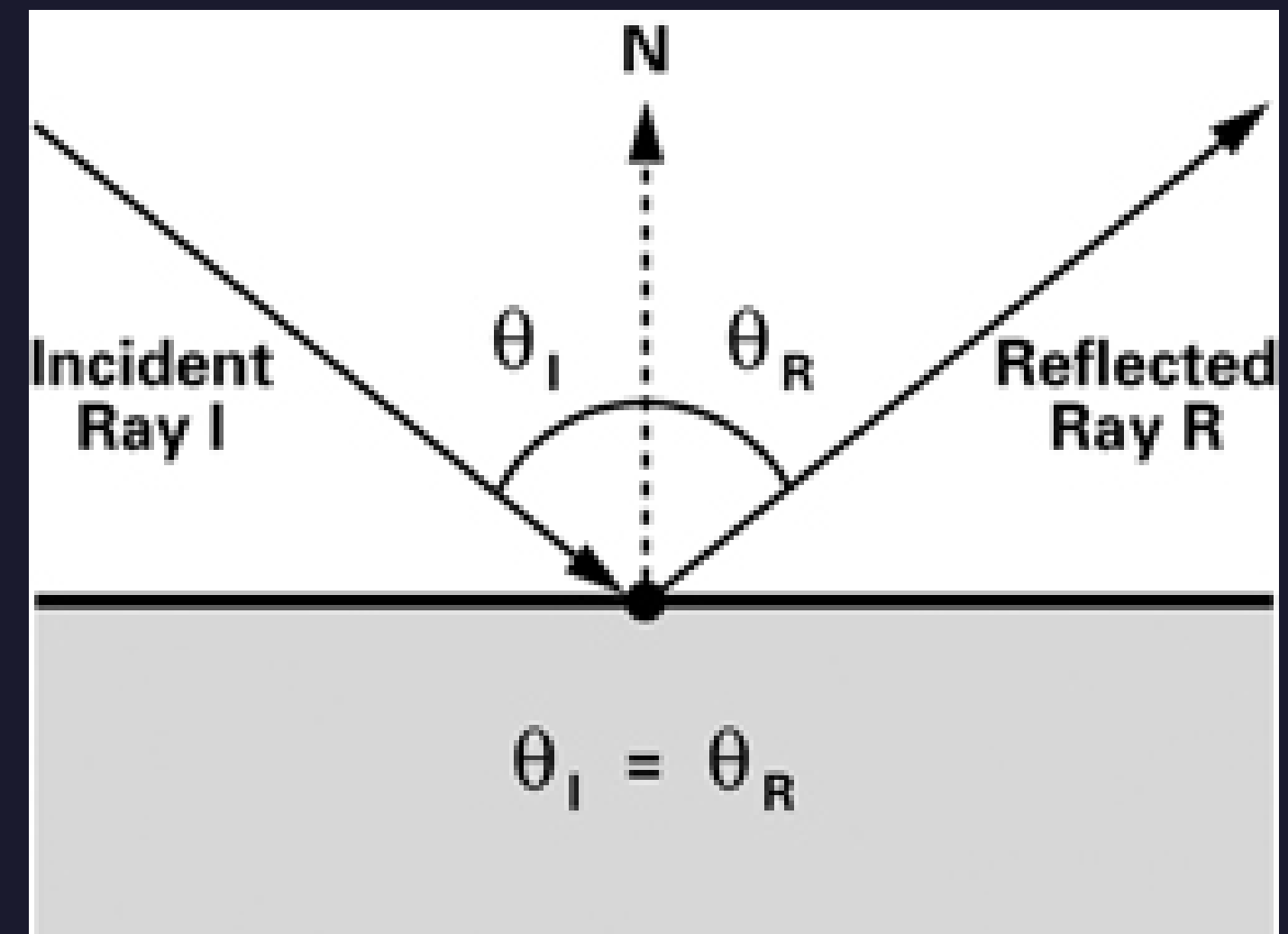
Se consideră *direcția de reflexie*, care realizează același unghi față de vectorul normal la suprafață pe care îl realizează vectorul pe direcția de vizualizare.

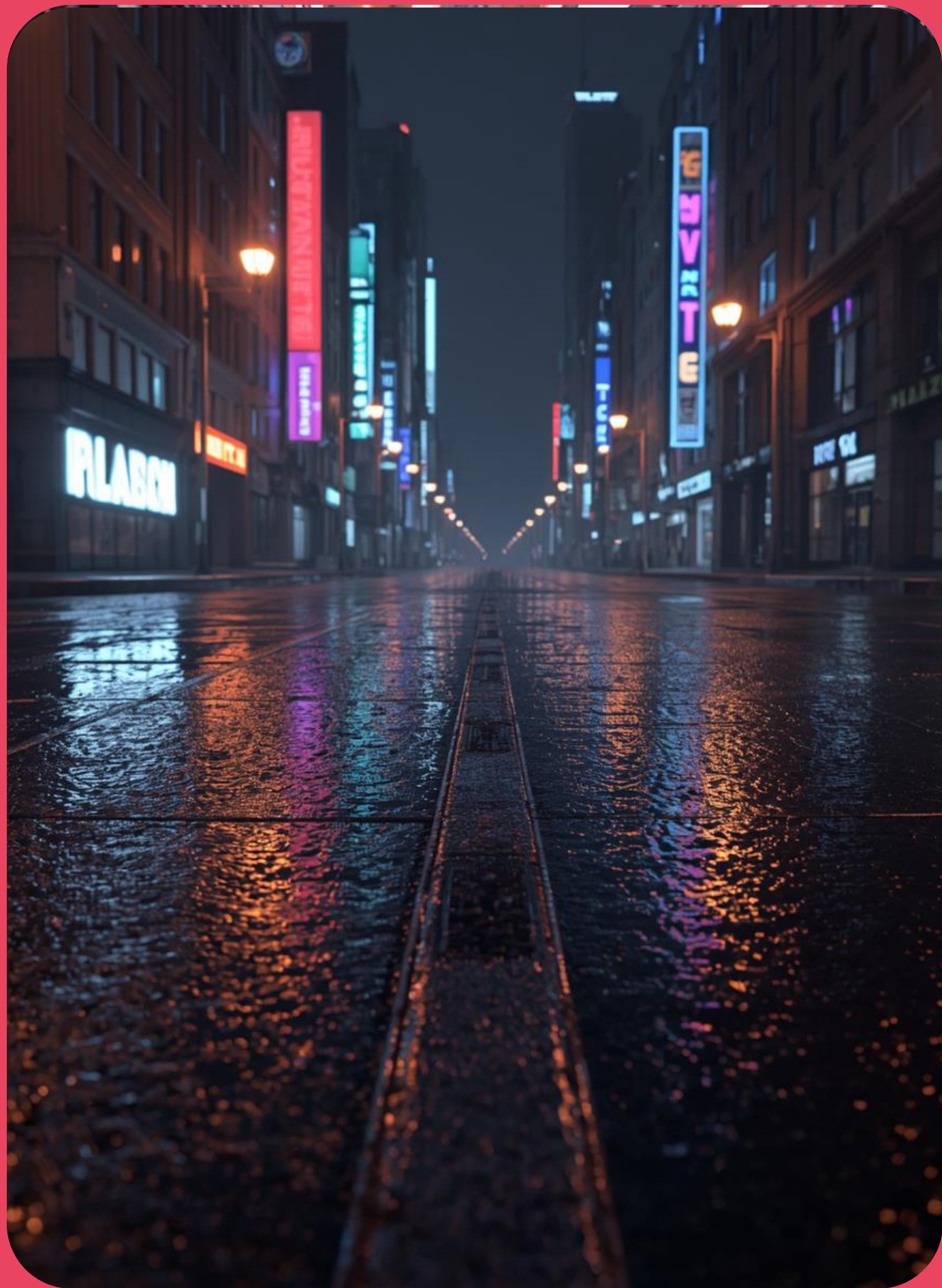




Cartografierea mediului înconjurător (V)

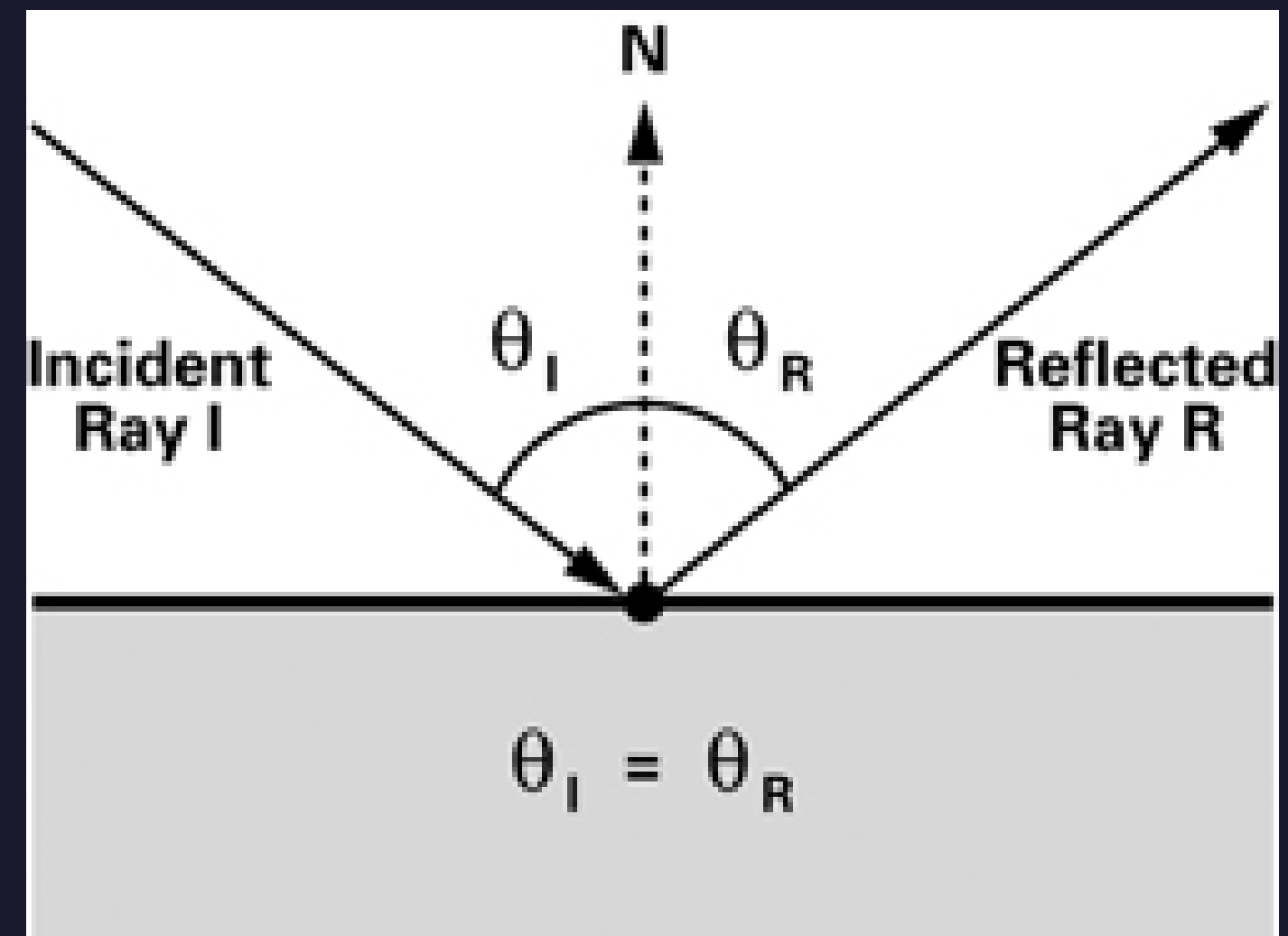
Se consideră *direcția de reflexie*, care realizează același unghi față de vectorul normal la suprafață pe care îl realizează vectorul pe direcția de vizualizare.

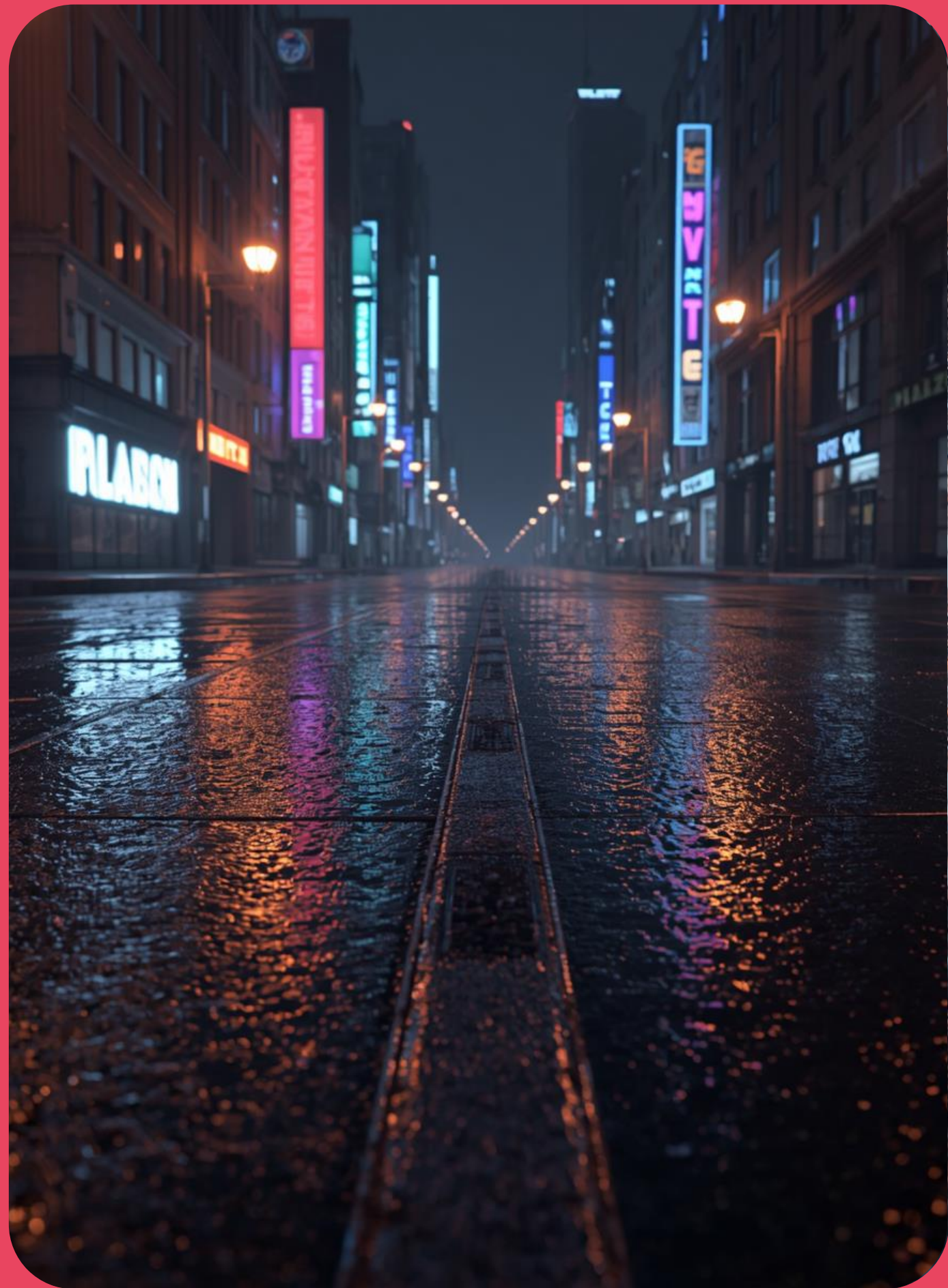




Cartografierea mediului înconjurător (VI)

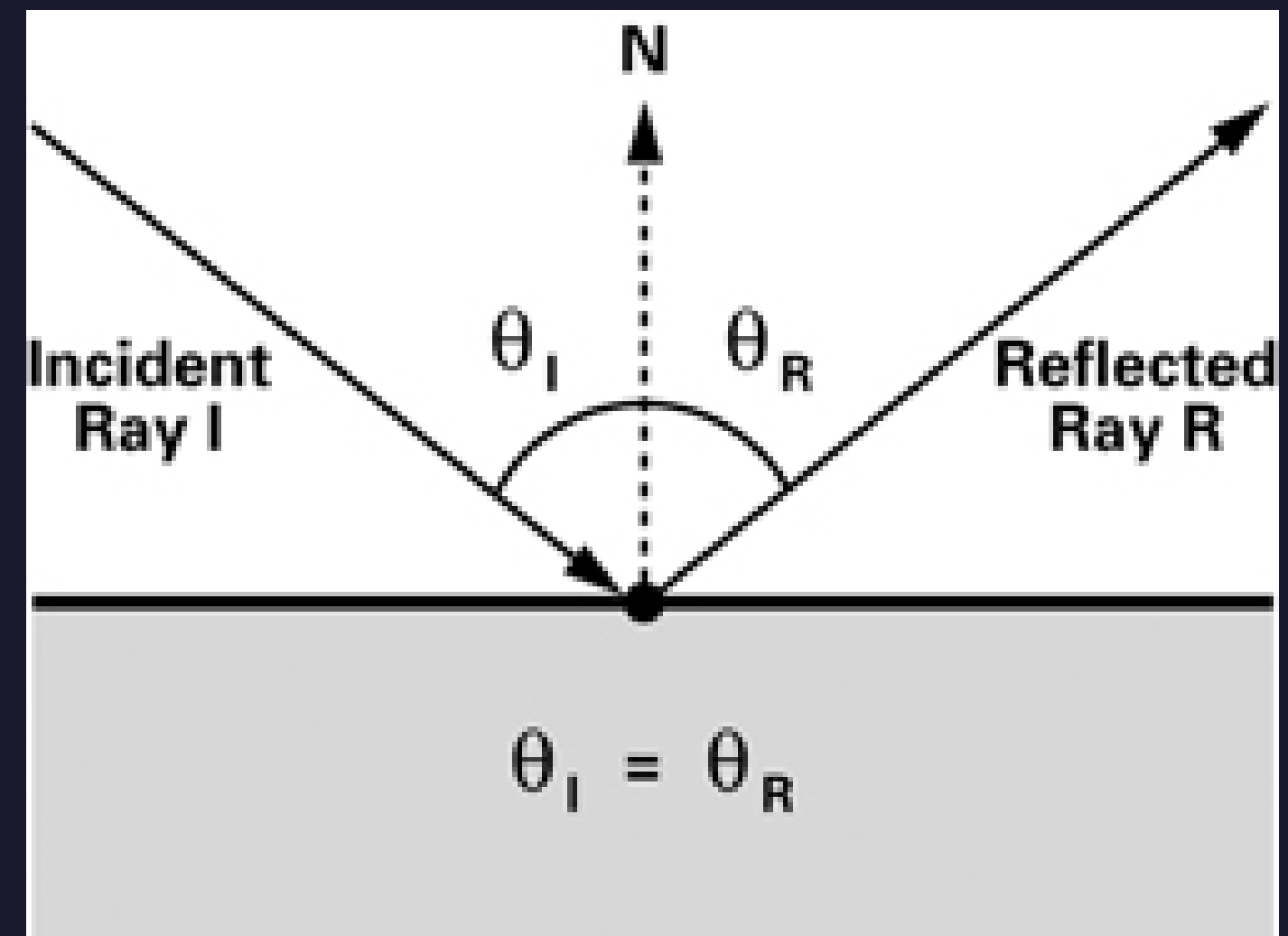
$$\vec{R} = \vec{I} - 2\vec{N}(\vec{N} \cdot \vec{I})$$

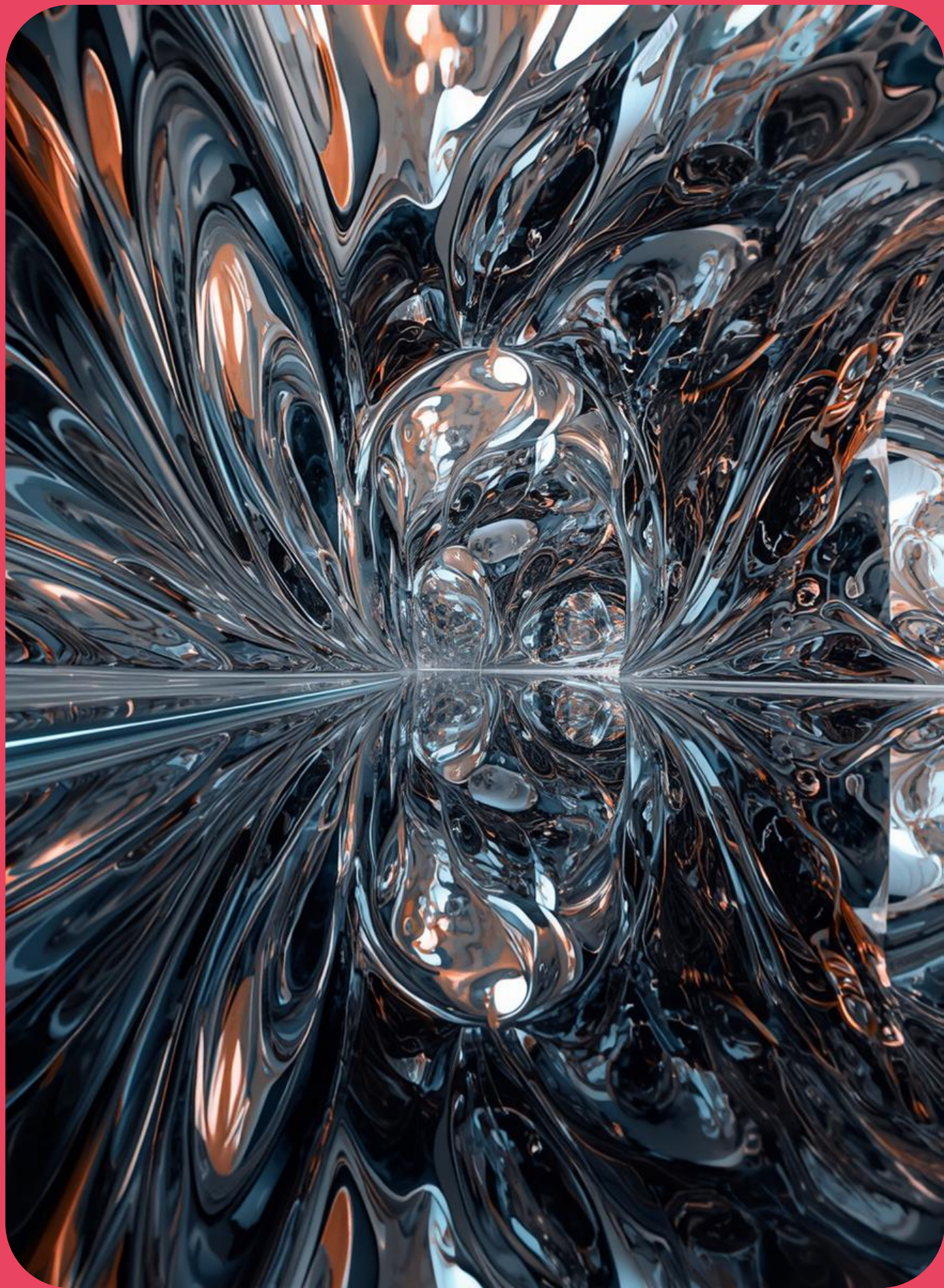




Cartografierea mediului înconjurător (VI)

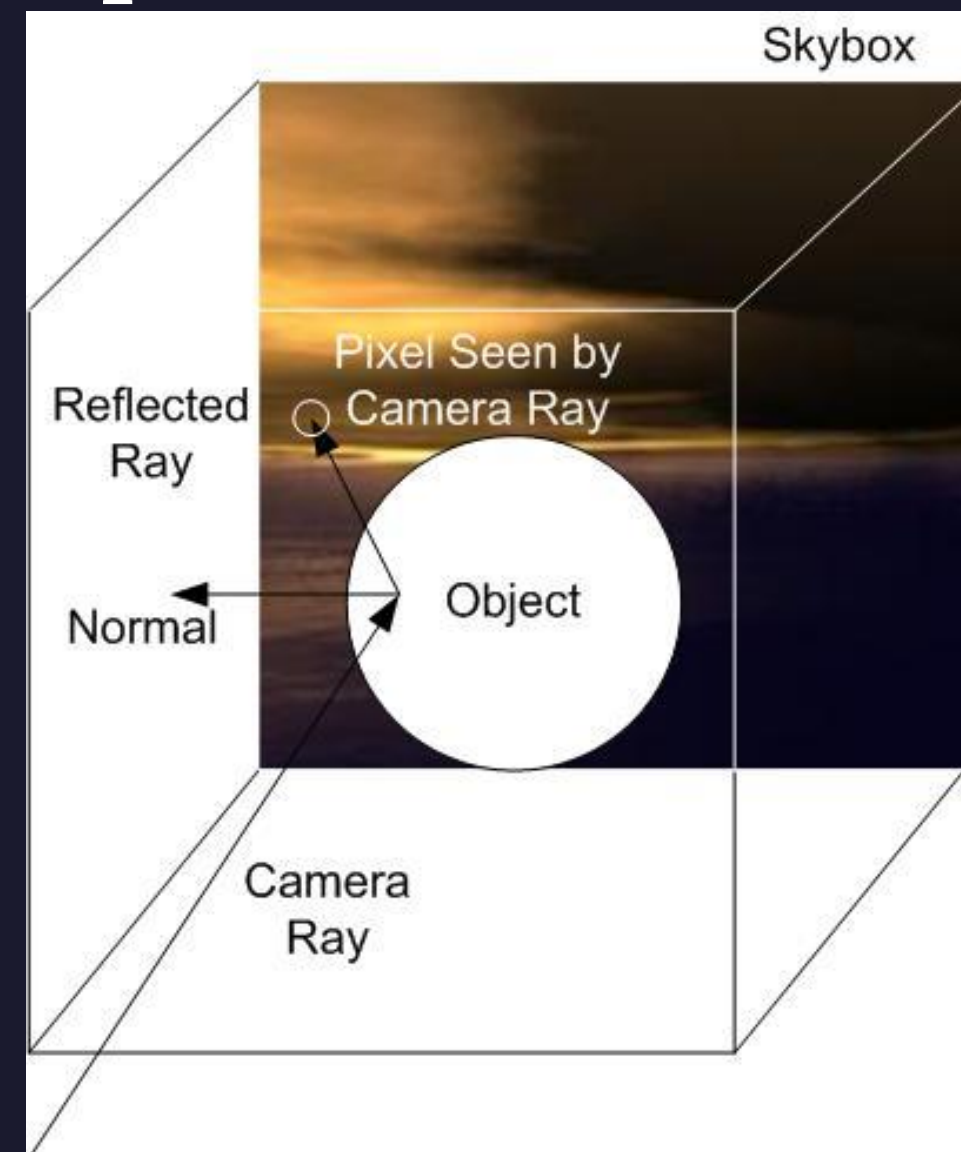
```
R = reflect(I, N);
```

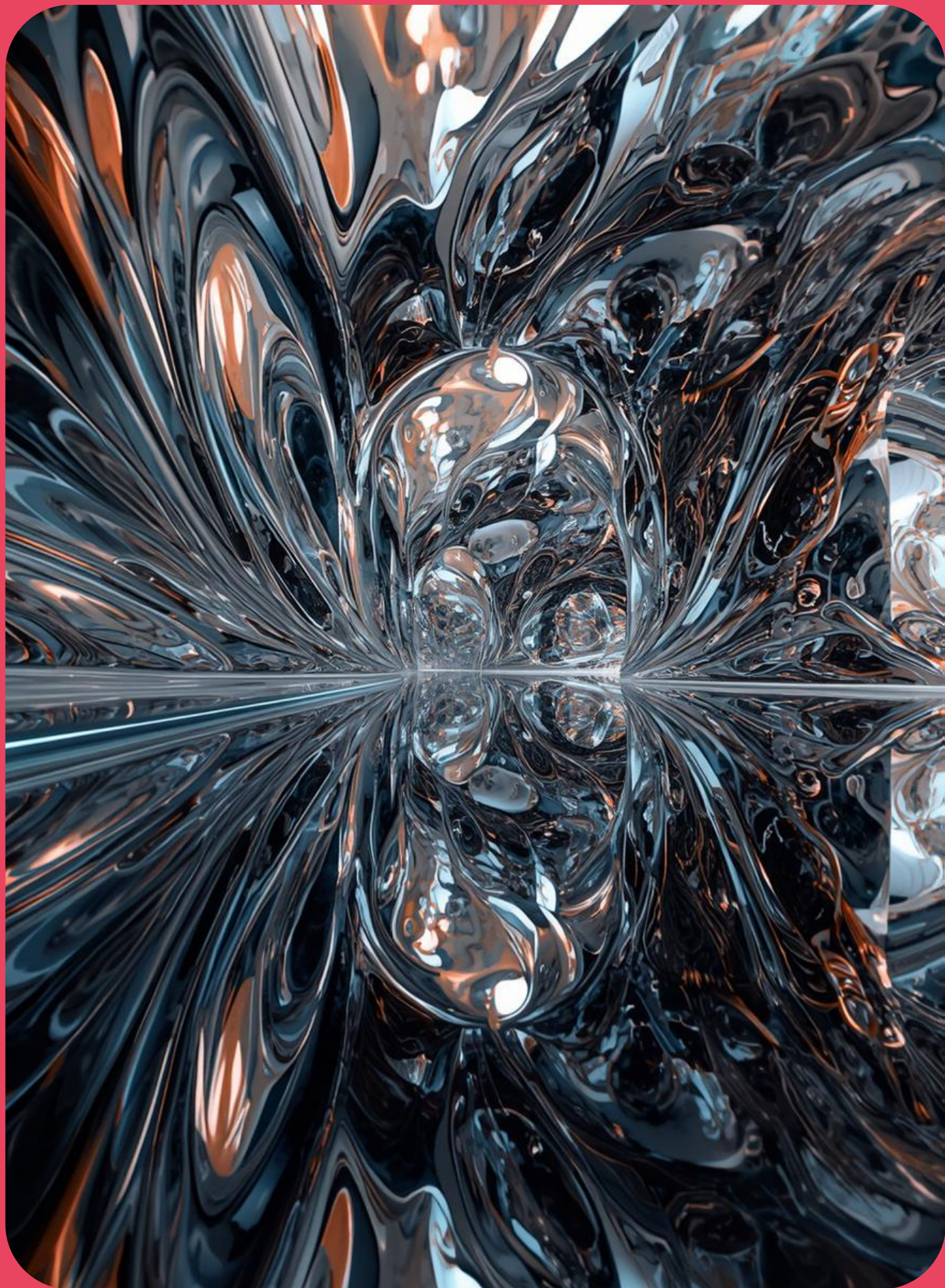




Cartografierea mediului înconjurător (VIII)

Pentru obținerea reflexiei, este necesară *intersecția razei pe direcția de reflexie* cu *fețele cubului* pe care este cartografiat obiectul de tip textură cubică.

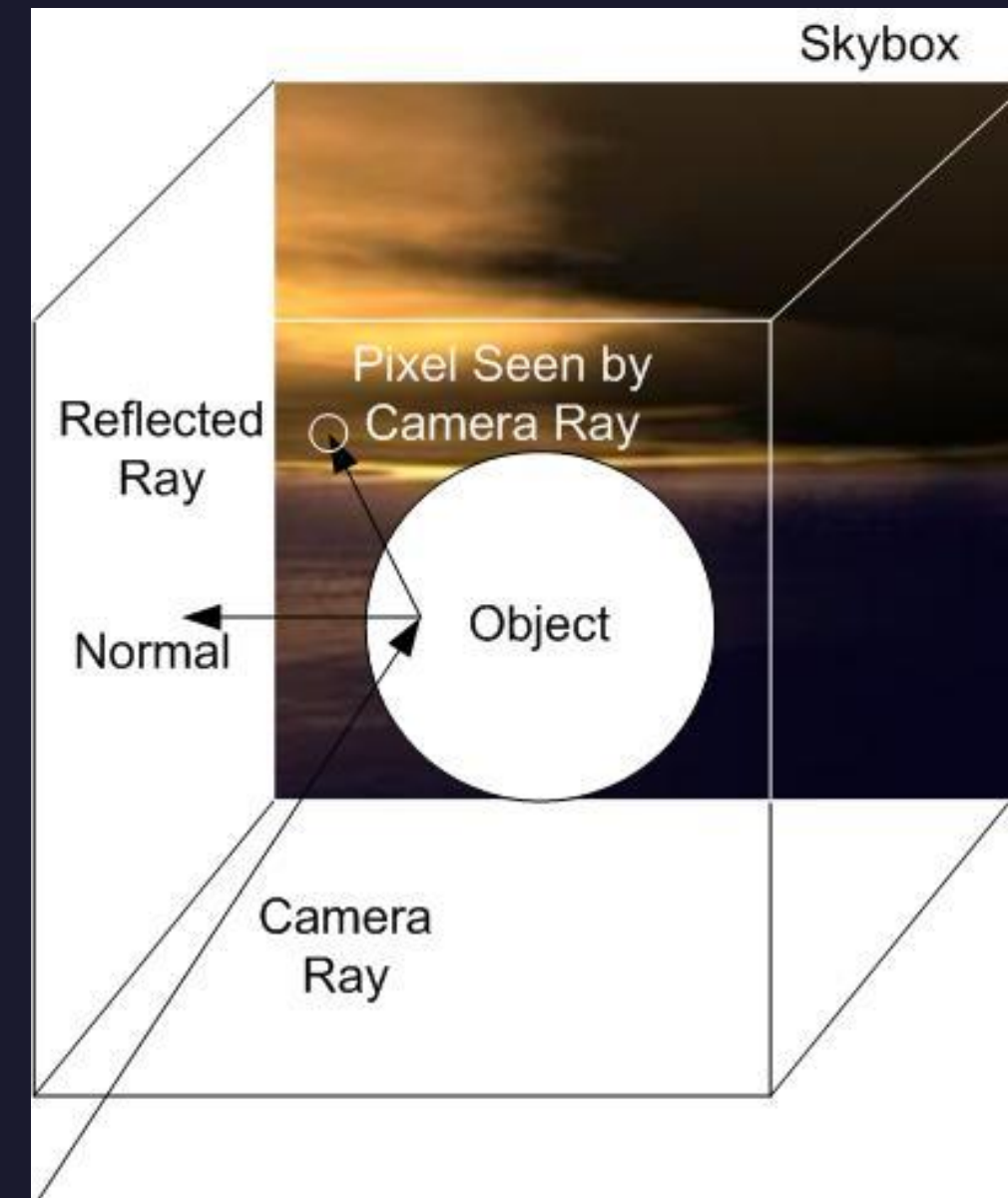




Cartografierea mediului înconjurător (IX)

```
uniform samplerCube cubeMap;
```

```
vec3 reflectedColor = texture(cubeMap, R);
```





Bibliografie

1. Akenine-Moller, Tomas, Eric Haines, and Naty Hoffman. Real-time rendering. AK Peters/crc Press, 2019.

Tehnici în spațiul ecranului (I)

Sunt, în general, efecte de *postprocesare*. Sunt efectuate *după ce se realizează calculele de iluminare* directă.

Se utilizează *informațiile geometrice în spațiul de vizualizare* împreună cu obiectul de tip textură care păstrează *culoarea obținută după calculele de iluminare*.



Tehnici în spațiul ecranului (II)

Prin utilizarea *informațiilor geometrice* disponibile, *se poate reconstrui parțial geometria scenei 3D*.

Această geometrie parțială a scenei 3D poartă numele de *câmp de lumină* - „*Light Field*”.



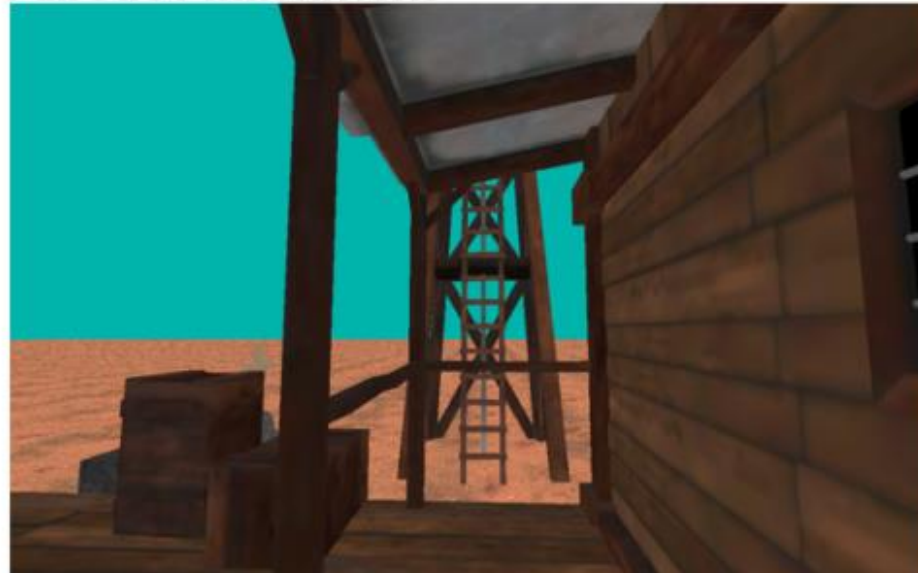
Tehnici în spațiul ecranului (III)

Se poate folosi geometria câmpului de lumină pentru realizarea unor metode ce utilizează geometria scenei. Chiar în situația în care **se utilizează doar această geometrie parțială** a scenei, disponibilă în câmpul de lumină, **se pot obține rezultate utile**.

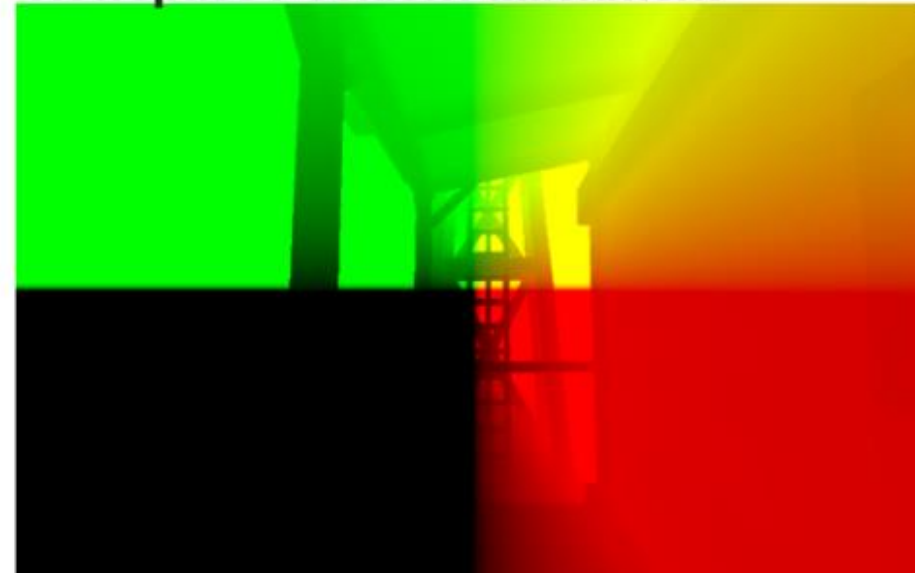
Deoarece acest câmp de lumină este obținut prin desenarea scenei din perspectiva observatorului și reprezintă informația de pe ecran, tehnicile care utilizează acest câmp de lumină sunt cunoscute sub numele de *tehnici în spațiul ecranului*.

Renderpass 0 : Gbuffer Pass

Albedo Attachment



Viewspace Position Attachment



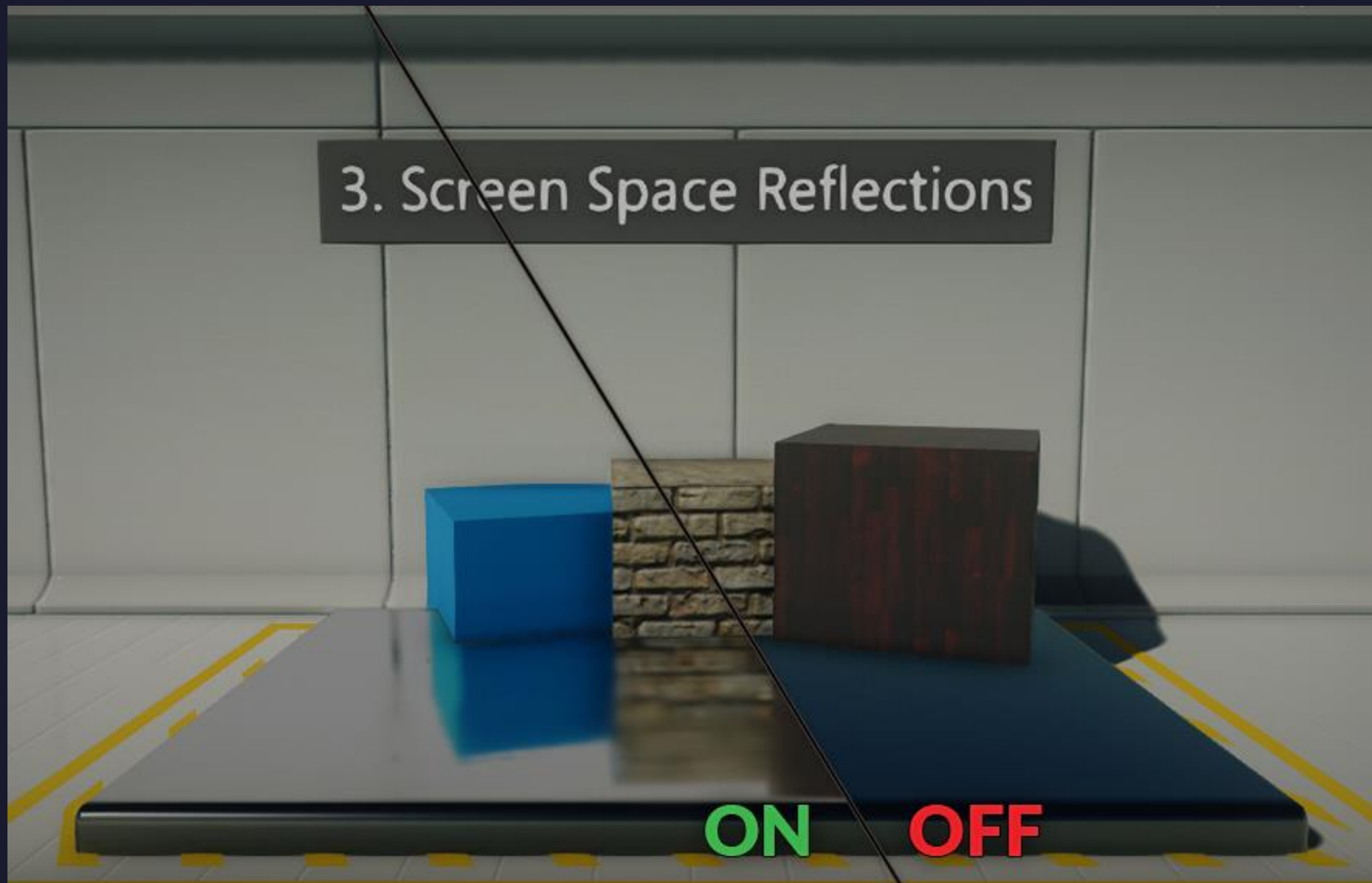


Tehnici în spațiul ecranului (IV)

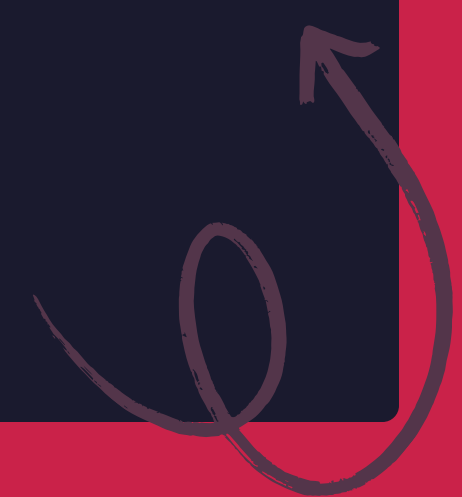
Unele dintre tehnicile ce utilizează informația din spațiul ecranului:

1. Obținerea de reflexii în spațiul ecranului - „*Screen Space Reflections*”;
2. Calcularea obturării componentei ambientale în spațiul ecranului - „*Screen Space Ambient Occlusion*”.

Obținerea reflexiilor în spațiul ecranului (I)



Obținerea reflexiilor în spațiul ecranului (II)



Obținerea reflexiilor în spațiul ecranului (III)



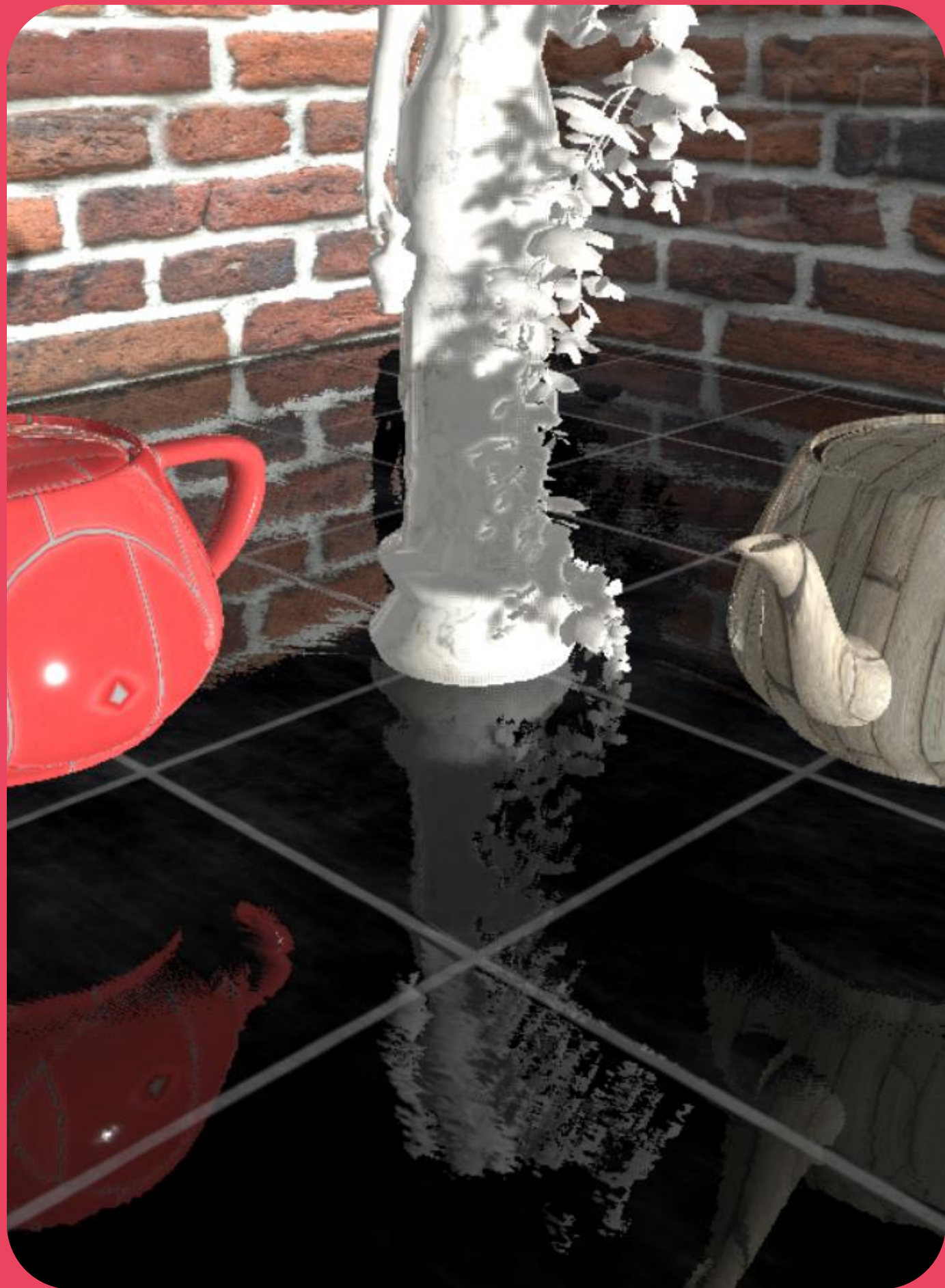


Obținerea reflexiilor în spațiul ecranului (IV)

Se realizeaza un proces de *aruncare a unei raze în câmpul de lumină* obținut prin desenarea scenei din perspectiva observatorului.

Se calculează *punctul de intersecție* a *razei pe direcția de reflexie* față de direcția de vizualizare cu *geometria prezentă în câmpul de lumină* obținut prin desenarea scenei din perspectiva observatorului.

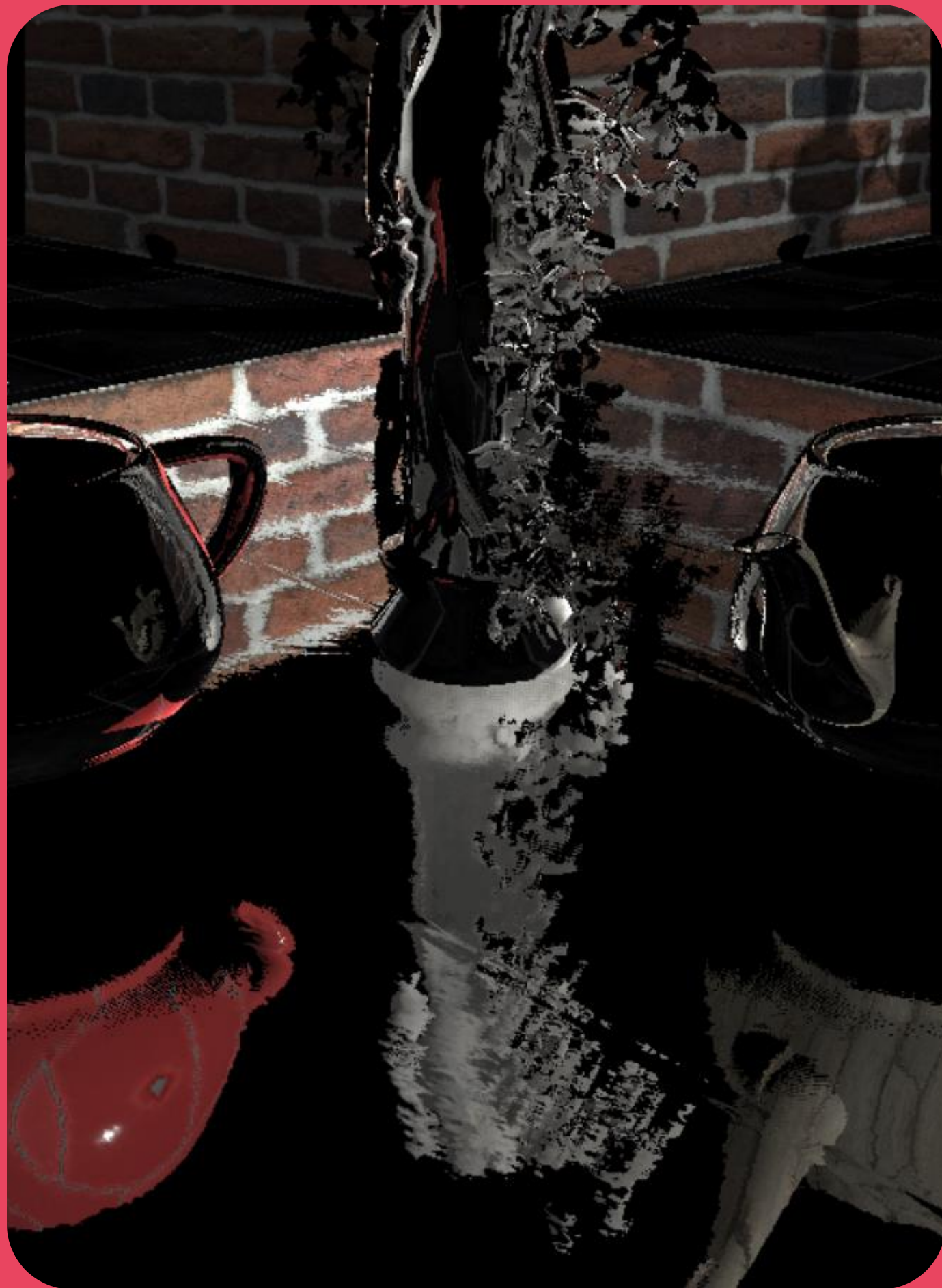
Calcularea punctului de intersecție se realizează prin *eșantionarea* obiectelor de tip textură care conțin *informatiile geometrice*.



Obținerea reflexiilor în spațiul ecranului (V)

Intersecția razei se poate realiza după cum urmează:

1. Se calculează *direcția razei de reflexie* pe suprafața desenată în fragment de la *direcția de vizualizare față de vectorul normal la suprafață*. Direcția de reflexie este în *spațiul de vizualizare*.
2. Se eșantionează un set de *puncte de-a lungul direcției de reflexie*, calculată anterior, puncte care se proiectează în spațiul ecranului.



Obținerea reflexiilor în spațiul ecranului (VI)

1. Se eșantionează obiectele de tip textură în care se pastreaza informațiile geometrice;
2. Se verifică faptul că punctul eșantionat se află în spatele geometriei câmpului de lumină din fragmentul eșantionat anterior.
3. În situația în care punctul eșantionat se află în spatele geometriei, la o distanță mică, se consideră că acesta este punctul de intersecție a razei pe direcția de reflexie cu geometria câmpului de lumină.

Obținerea reflexiilor în spațiul ecranului (VII)

```
vec3 SSR(vec3 viewPosition, vec3 viewNormal)
{
    vec3 viewDirection = normalize(-viewPosition);
    vec3 reflectionDirection = reflect(viewDirection, viewNormal);

    float viewSpaceStepSize = 0.1f;
    float maxDistance = 10.0;

    float traveled = 0.0;
    vec3 rayPosition = viewPosition;

    while (traveled < maxDistance) {
        rayPosition += reflectionDirection * viewSpaceStepSize;
        traveled += viewSpaceStepSize;

        vec4 clipSpacePosition = projectionMatrix * vec4(rayPosition, 1.0f);

        vec3 screenSpacePosition = clipSpacePosition.xyz / clipSpacePosition.w;
        screenSpacePosition = screenSpacePosition * 0.5f + 0.5f;
    }
```



Obținerea reflexiilor în spațiul ecranului (VIII)

```
vec3 screenSpacePosition = clipSpacePosition.xyz / clipSpacePosition.w;
screenSpacePosition = screenSpacePosition * 0.5f + 0.5f;

vec2 uv = screenSpacePosition.xy;
float sampleDepth = screenSpacePosition.z;

if (uv.x < 0.0 || uv.x > 1.0 || uv.y < 0.0 || uv.y > 1.0) {
    break;
}

float sceneDepth = texture(depthTexture, uv).r;

if (sampleDepth > sceneDepth) {
    return texture(colorTexture, uv).rgb;
}

return vec3(0.0);
}
```



Obținerea reflexiilor în spațiul ecranului (IX)

Pentru îmbunătățirea rezultatelor vizuale, se poate utiliza o eșantionare a razei din spațiul ecranului, în locul eșantionării razei în spațiul de vizualizare.



McGuire, Morgan, and Michael Mara. "Efficient GPU screen-space ray tracing." *Journal of Computer Graphics Techniques (JCGT)* 3.4 (2014): 73-85.

Obținerea reflexiilor în spațiul ecranului (X)

Pentru **optimizarea** procesului de eșantionare a setului de puncte de-a lungul razei aflată pe direcția de reflexie, se poate realiza un proces de „**căutare binară**” a **punctului de intersecție**.

Se utilizează o **piramidă „mipmap”** specială pentru **obiectul de tip textură care conține valori de adâncime**. În piramidă, la un nivel i , se stochează valoarea maximă de adâncime de la nivelul $i-1$.

În momentul în care se eșantionează setul de puncte pe direcția de reflexie, **se poate sări peste un set de puncte** pe baza informației de intersecție stocate în nivelurile piramidei „mipmap”.

Hierarchical depth buffers Hierarchical Depth Buffers - Mike Turitzin. Available at:
<https://miketuritzin.com/post/hierarchical-depth-buffers/> (Accessed: 08 April 2026).

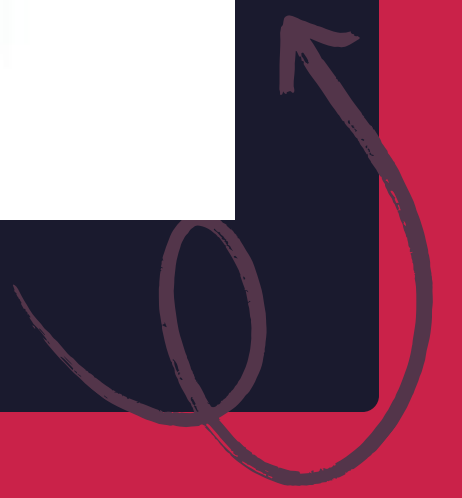


Bibliografie

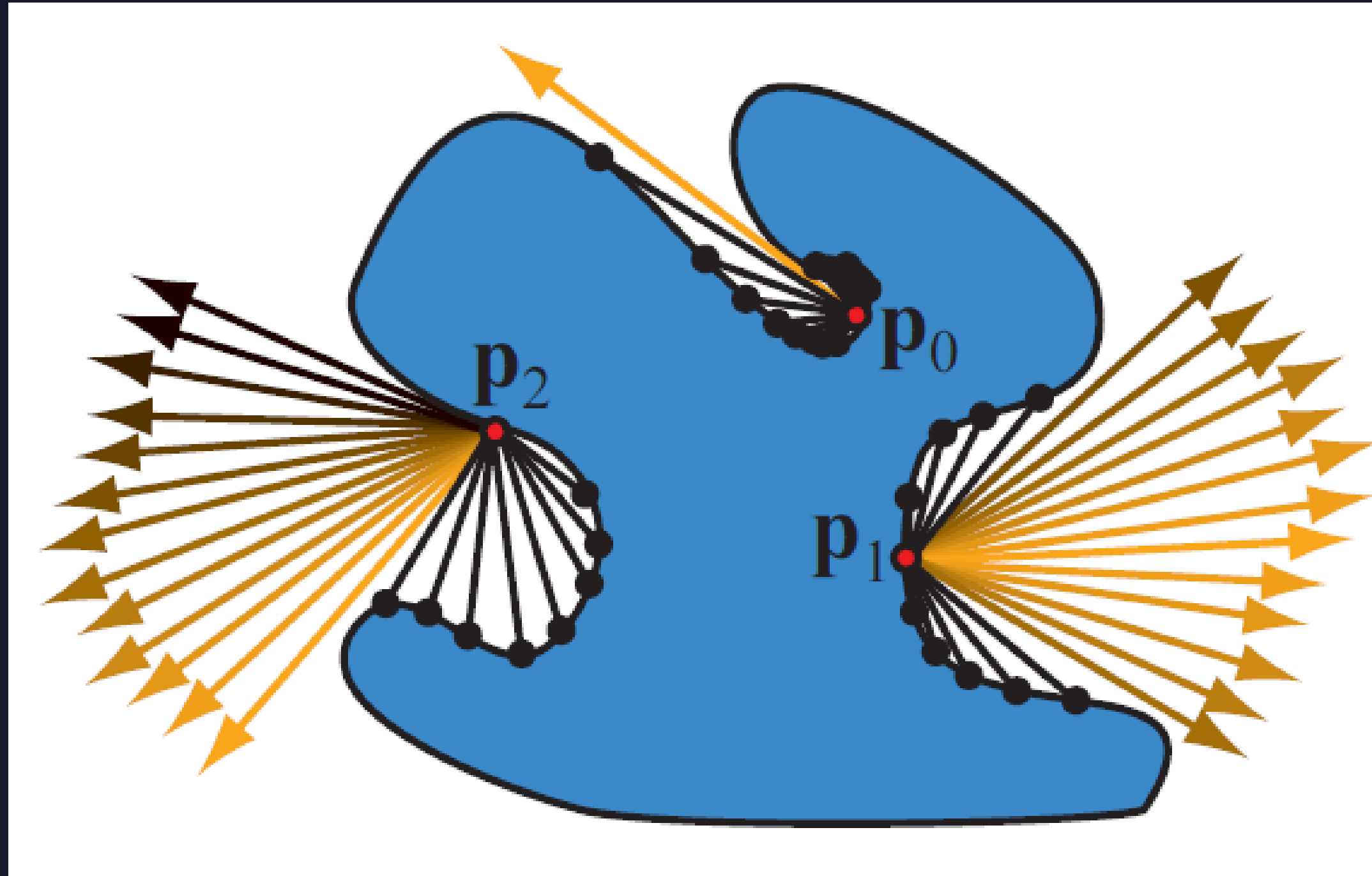


1. Rtroe (2018) Shader Series – Screen Space Reflections, Virtex Edge Design. Available at: <https://virtexedgedesign.wordpress.com/2018/06/24/shader-series-basic-screen-space-reflections/> (Accessed: 08 April 2026).
2. Dongen, J. van Screen space reflections in Blightbound, Screen Space Reflections in Blightbound. Available at: <https://joostdevblog.blogspot.com/2020/10/screen-space-reflections-in-blightbound.html> (Accessed: 08 April 2026).
3. Lettier, D. (no date) 3D game shaders for Beginners, Screen Space Reflection. Available at: <https://lettier.github.io/3d-game-shaders-for-beginners/screen-space-reflection.html> (Accessed: 08 April 2026).

Obturarea componentei ambientale (I)

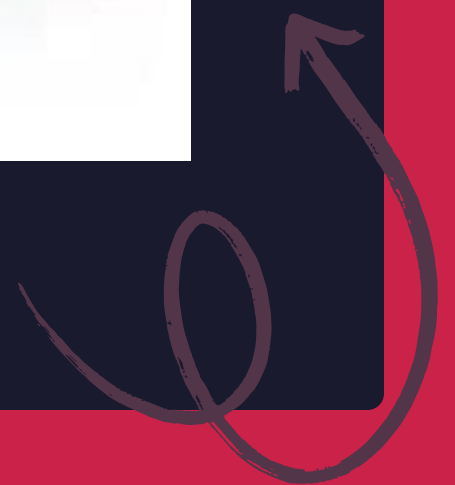
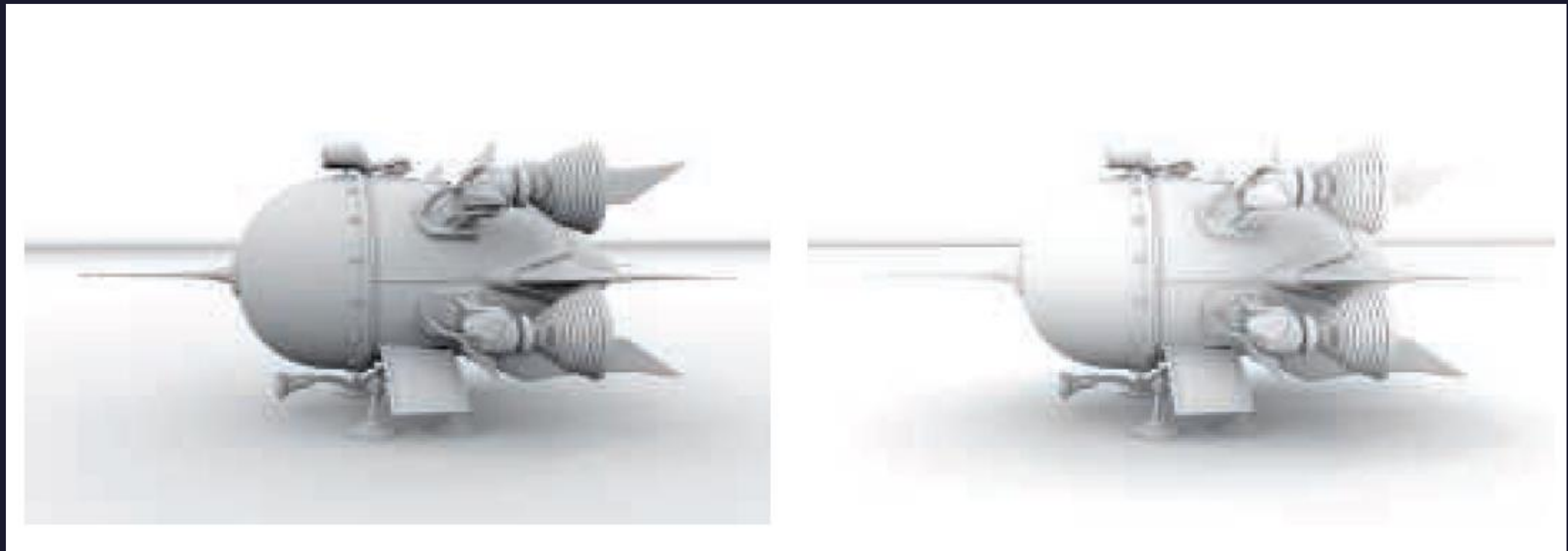


Obturarea componentei ambientale (II)



Obturarea componentei ambientale (III)

Limitarea distanței de calculare a geometriei care obturează componenta ambientală

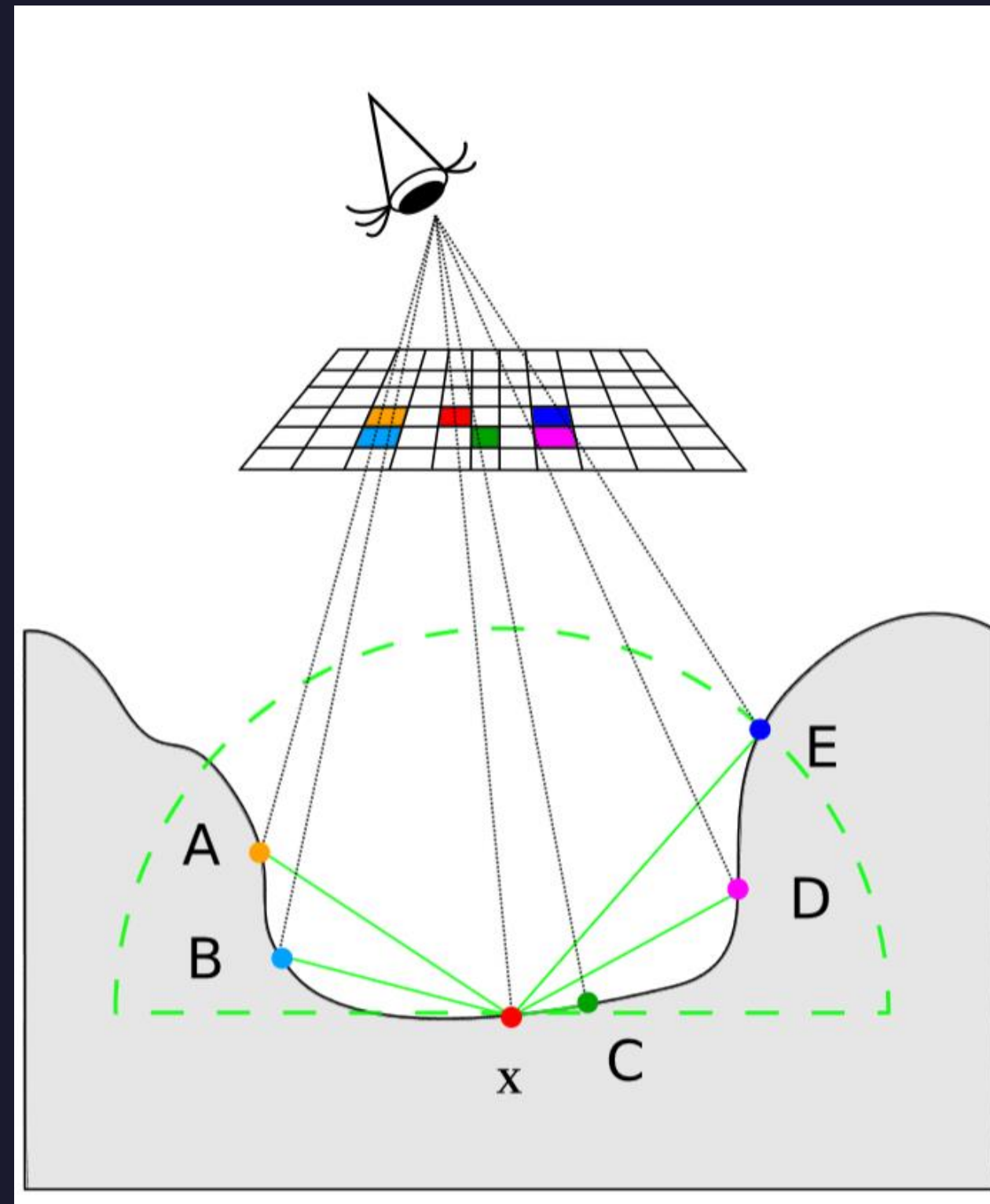


Obturarea componentei ambientale (IV)

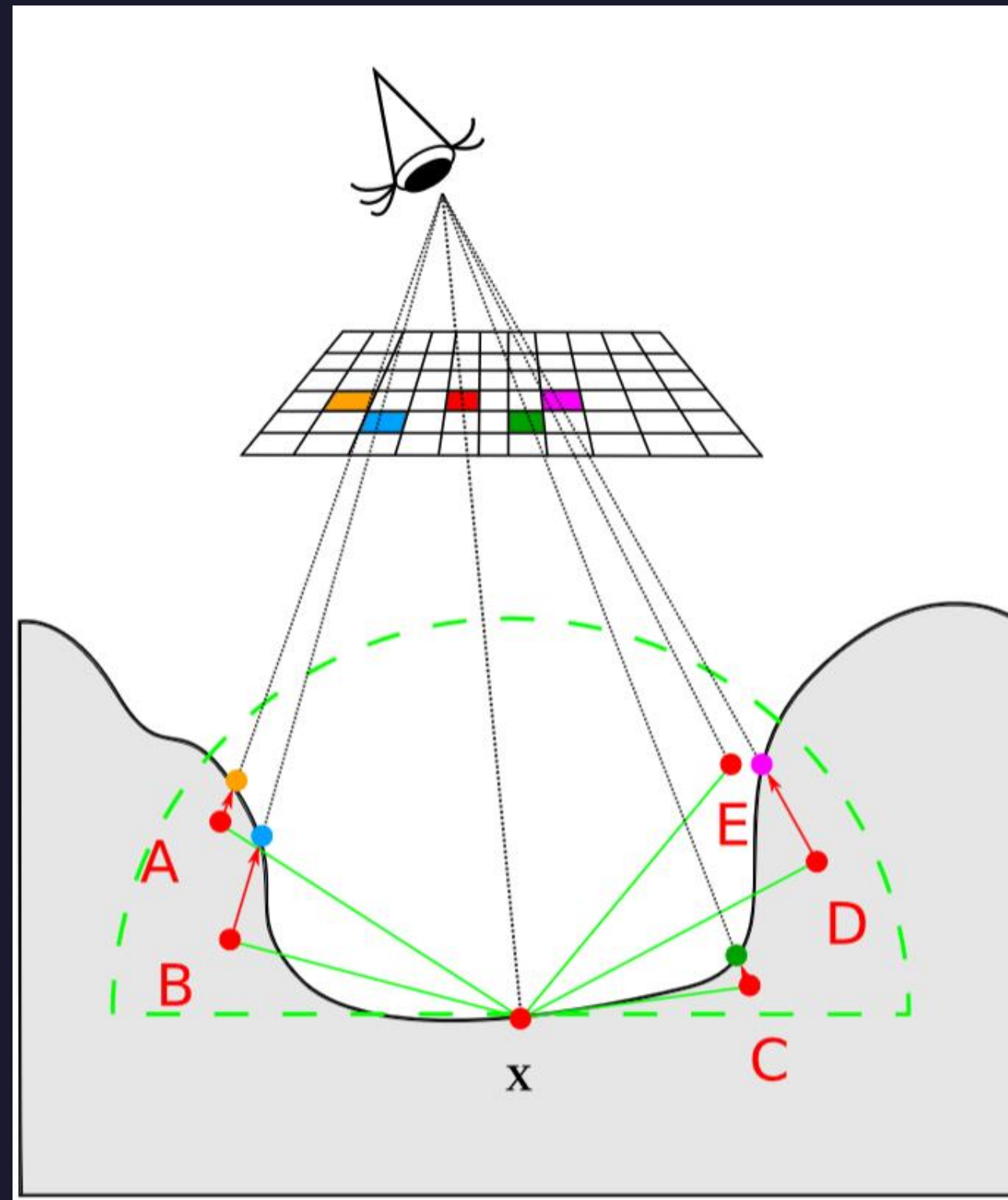
Limitarea distanței de calculare a geometriei care obturează componenta ambientală



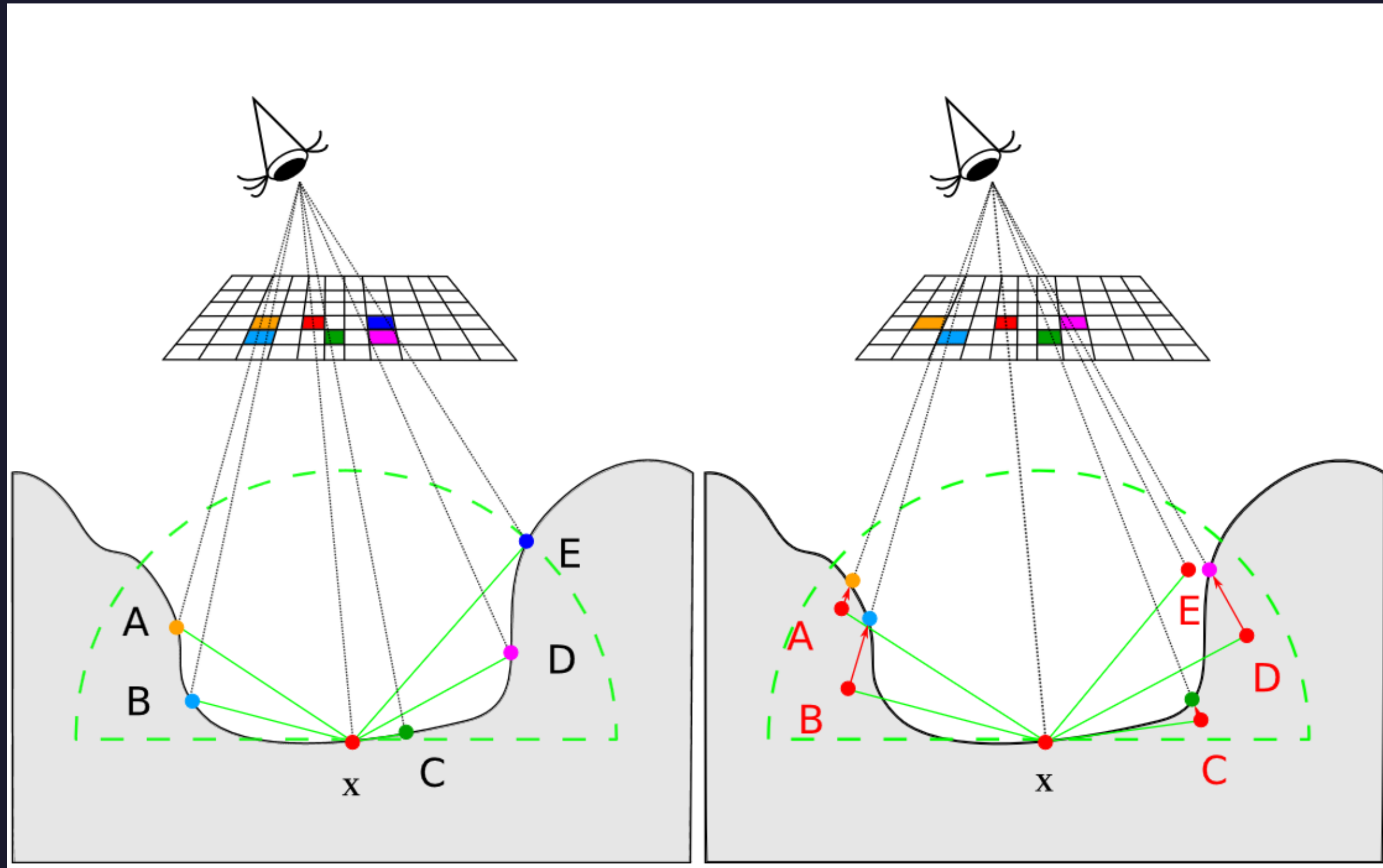
Obturarea componentei ambientale în spațiul ecranului (I)

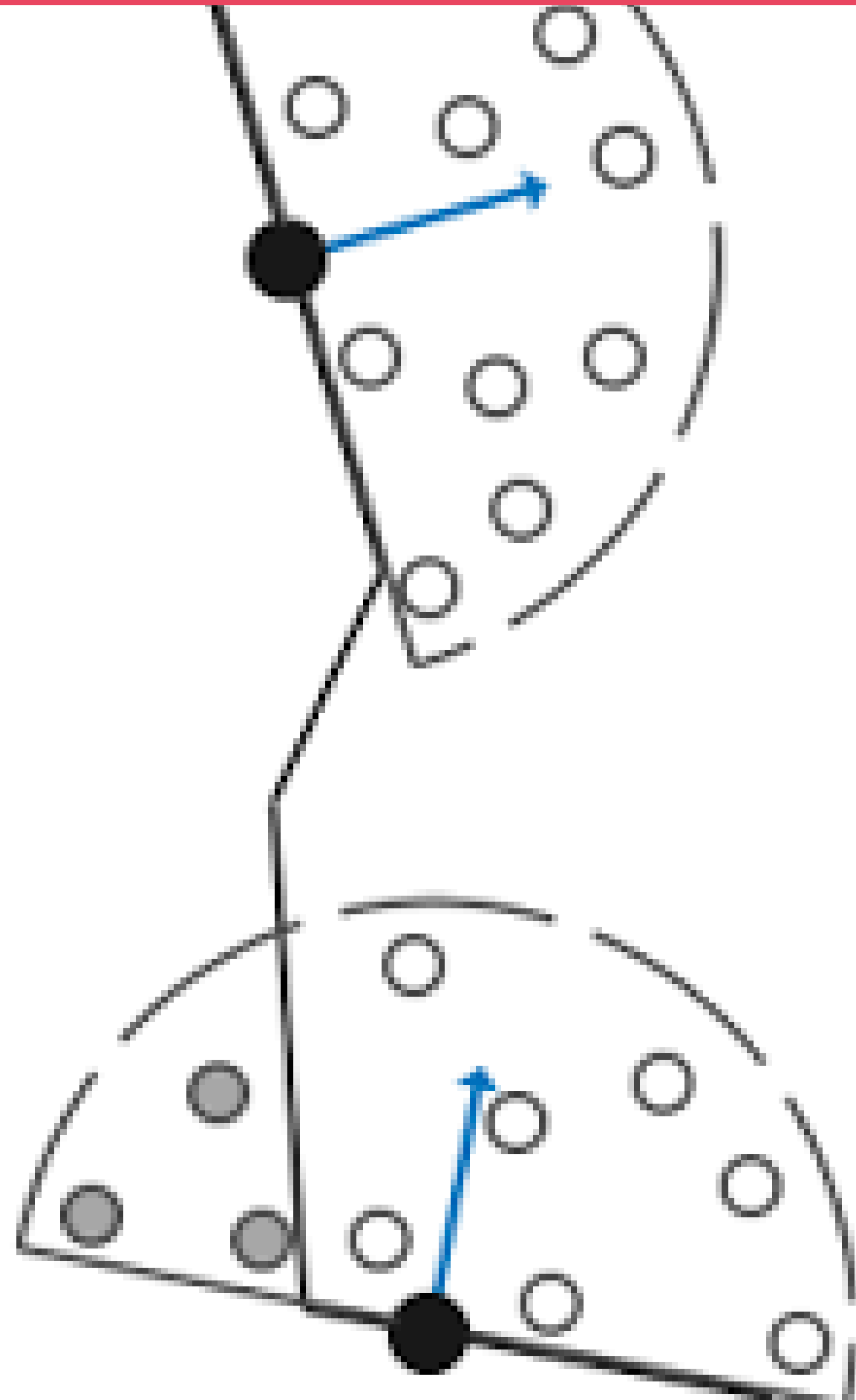


Obturarea componentei ambientale în spațiul ecranului (II)



Obturarea componentei ambientale în spațiul ecranului (III)





Obturarea componentei ambientale în spațiul ecranului (IV)

Se eșantionează *un set de puncte* în emisfera aflată în direcția vectorului normal la suprafață:

- Se calculează *proiecția punctului în spațiul ecranului*;
- Se *eșantionează informațiile geometrice* din obiectele de tip textură *la poziția punctului proiectat* în spațiul ecranului;
- Se numără *punctele proiectate* care *sunt în fața punctului eșantionat*;
- Procentul de puncte proiectate care sunt în față/numarul total de puncte dă *valoarea de ponderare* pentru obturarea componentei ambientale.

Obturarea componentei ambientale în spațiul ecranului (V)

```
float SSAO(vec3 viewPosition, vec3 viewNormal)
{
    float viewSpaceRadius = 0.5;
    float bias = 0.025;
    float occlusion = 0.0;

    for (int i = 0; i < 16; i++) {
        vec3 viewSpaceSamplePosition = viewPosition + samples[i] * viewSpaceRadius;

        vec4 clipSpacePosition = projectionMatrix * vec4(viewSpaceSamplePosition, 1.0);
        vec3 ndc = clipSpacePosition.xyz / clipSpacePosition.w;
        vec2 uv = ndc.xy * 0.5 + 0.5;

        if (uv.x < 0.0 || uv.x > 1.0 || uv.y < 0.0 || uv.y > 1.0) {
            continue;
        }

        float sampleDepth = ndc.z * 0.5 + 0.5;
        float sceneDepth = texture(depthTexture, uv).r;

        if (sampleDepth > sceneDepth + bias) {
            occlusion += 1.0;
        }
    }

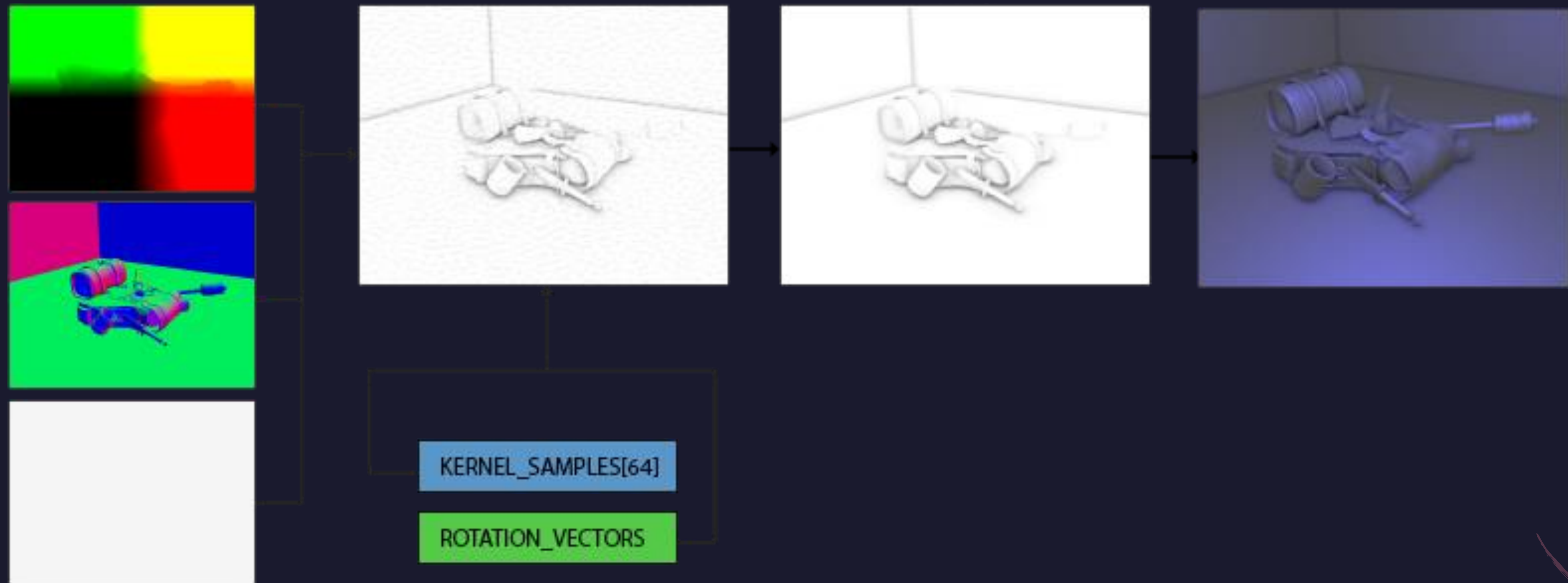
    occlusion = 1.0 - (occlusion / 16.0);
    return occlusion;
}
```



Obturarea componentei ambientale în spațiul ecranului (VI)

Diferite ajustări:

- Rotații aleatorii ale setului de eșantioane în jurul vectorului normal la suprafață;
- Utilizarea unui efect de netezire aplicat asupra rezultatului de SSAO.





Bibliografie

1. Akenine-Moller, Tomas, Eric Haines, and Naty Hoffman. Real-time rendering. AK Peters/crc Press, 2019.



Recomandări de proiecte (I)

Echipe de o persoană:

- Cartografierea tonurilor de culoare - „*Tone Mapping*” prin utilizarea unei plaje dinamice de intensități pentru iluminare - „*High Dynamic Range*”;
- Hărți de culori - „*Tone Mapping Lookup Table*”;
- Efectul de tip „*bloom*”;
- Alte efecte de postprocesare, de complexitate mică, alese de studenți.



Recomandări de proiecte (II)

Echipe de 2 persoane:

- Efectul de adâncime a câmpului vizual - „*Depth of Field*”;
- Efectul de iluminare volumetrică - „*Volumetric Illumination*”;
- Alte efecte de postprocesare, de complexitate medie, alese de studenți.



Recomandări de proiecte (III)

Echipe de 3 persoane:

- Combinație de mai multe efecte de postprocesare;
- Alte efecte de postprocesare, de complexitate mare, alese de studenți;
- Obținerea reflexiilor în spațiul ecranului - „*Screen Space Reflections*”.
- Obținerea componentei ambientale în spațiul ecranului - „*Screen Space Ambient Occlusion*”.